Kunming AI+

An Applied Introduction to AI with PyTorch

Luca Ghafourpour

UNIVERSITY OF CAMBRIDGE
Department of Engineering

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance + *Weights and Biases*
Practical Workshop 3	Initialisation and regularisation
Workshop 4	Convolutional neural networks
Practical Workshop 4	**Dimensionality reduction, k-fold CV, confusion matrices**
Academic Lecture 1	Physics Informed Neural Networks
Academic Lecture 2	Neural Operators
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

Today

1. **Dimensionality reduction**

   Correlations, PCA, and feature importance

2. **Evaluation that truly checks generalisation capabilities**

   Train / validation / test discipline and cross-validation.

3. **Keeping it fair**

   Data leakage.

4. **Better metrics of model performancs**

   Confusion matrices, precision / recall.

Where this session stands

The course so far has been about building and optimising models.
Two questions remain for any model you train:

Is it any good?

How well does our model really generalise?

What is it doing?

Which inputs matter, and what structure has it found?

Dimensionality reduction

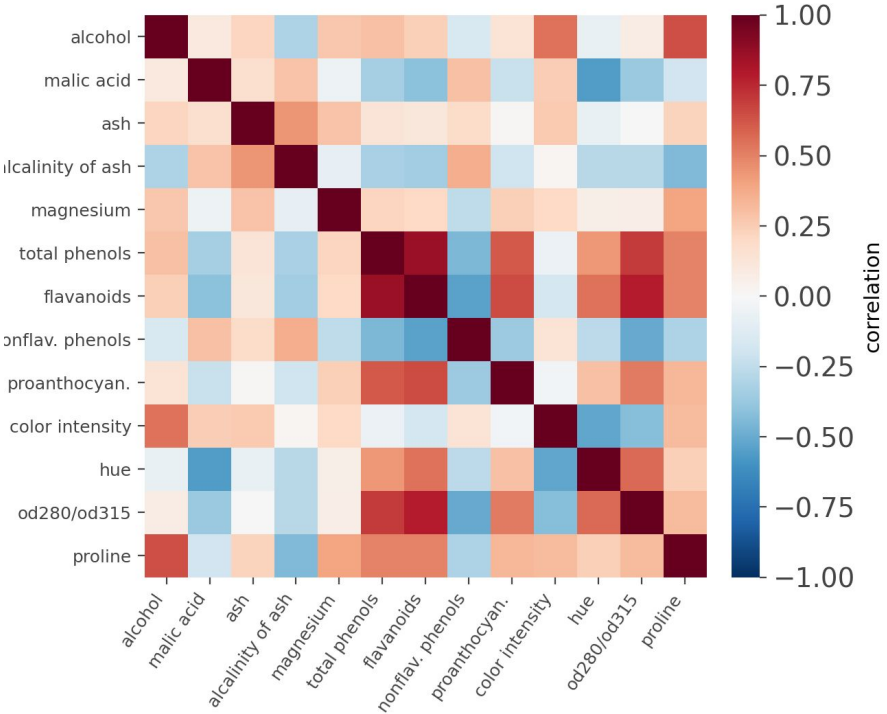
Correlation analysis

Before modelling, look at how features relate. A correlation matrix shows which ones move together.

Consider collecting a dataset from a sample of adults, where we want to measure two features F_1 and F_2, for example, weight (in kg) and height (in cm). We define the correlation between F_1 and F_2 as:

$$\rho_{F_1,F_2} = \frac{\text{Covariance}(F_1, F_2)}{\sigma_{F_1} \cdot \sigma_{F_2}}$$

Correlation analysis

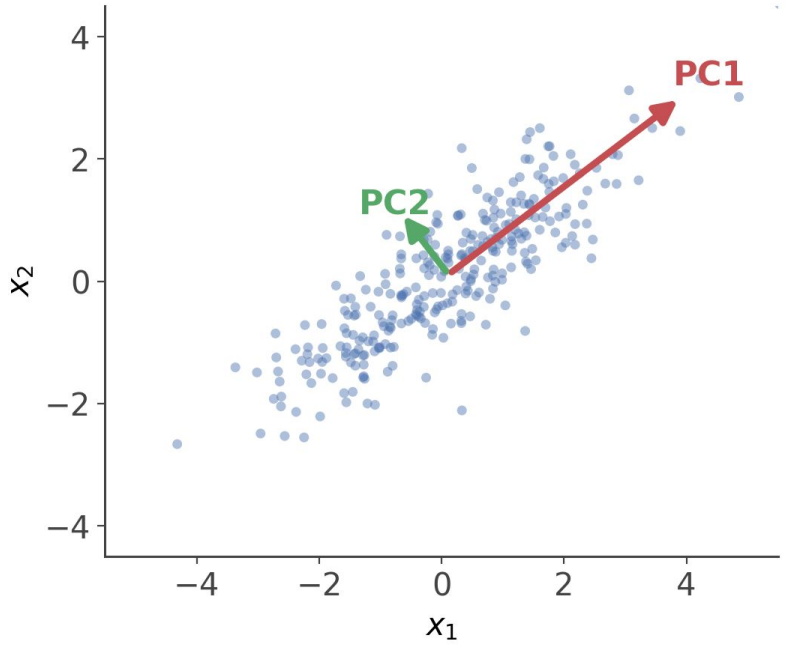


For our machine learning models to capture patterns of interest, we would like that our dataset contains a lot of variability.

Strong correlations between features mean redundancy, where, several features carry much the same information.

Correlated features add little new information, and as such we can discard some features.

Principal Component Analysis (PCA)



Real datasets contain much of their variation into a few directions. PCA finds new orthogonal axes (principal components) ordered by the variance each captures.

The data forms an ellipsoid; PC1 is its longest axis (greatest variance), PC2 the next, orthogonal to it.

When features are correlated the axes are tilted: each component is a linear combination of the original features.

Solution to PCA Optimisation Problem

We can write the PCA optimisation problem as the following:

$$\text{Find } \mathbf{w} \text{ subject to } \|\mathbf{w}\|^2 = c \text{ such that}$$

$$\text{Var}_{\mathbf{w}} = \frac{1}{N-1}\sum_{t=1}^{N}[(\mathbf{x}^{(t)} - \bar{\mathbf{x}}) \cdot \mathbf{w}]^2$$

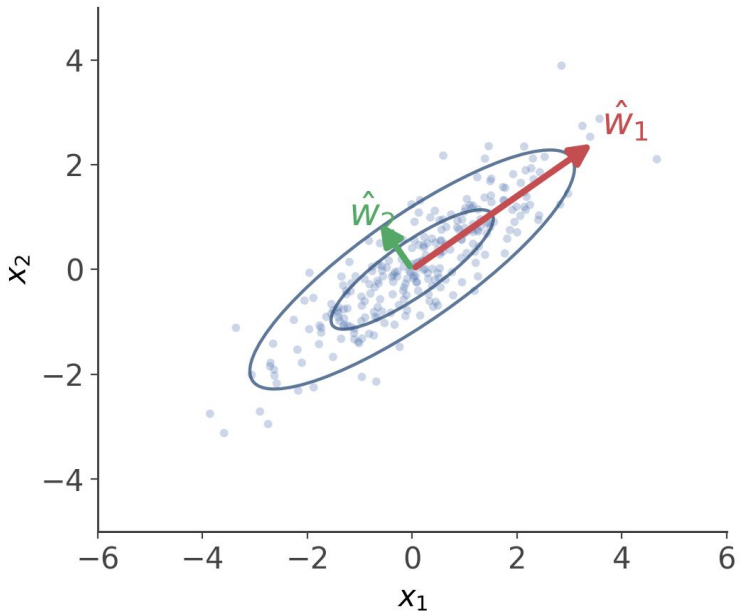
$$\text{is maximised.}$$

The solution of the problem is found by solving the following linear system:

$$\Sigma\hat{\mathbf{w}}_i = \lambda_i\hat{\mathbf{w}}_i \quad i = 1, 2, \ldots \text{rank}(\Sigma)$$

Hence, the principal components are the unit eigenvectors of the covariance matrix Σ !

PCA Algorithm



The variance along each principal component is its eigenvalue, with $\lambda_1 \geq \lambda_2 \geq \ldots$ (all ≥ 0; the axes are orthogonal).

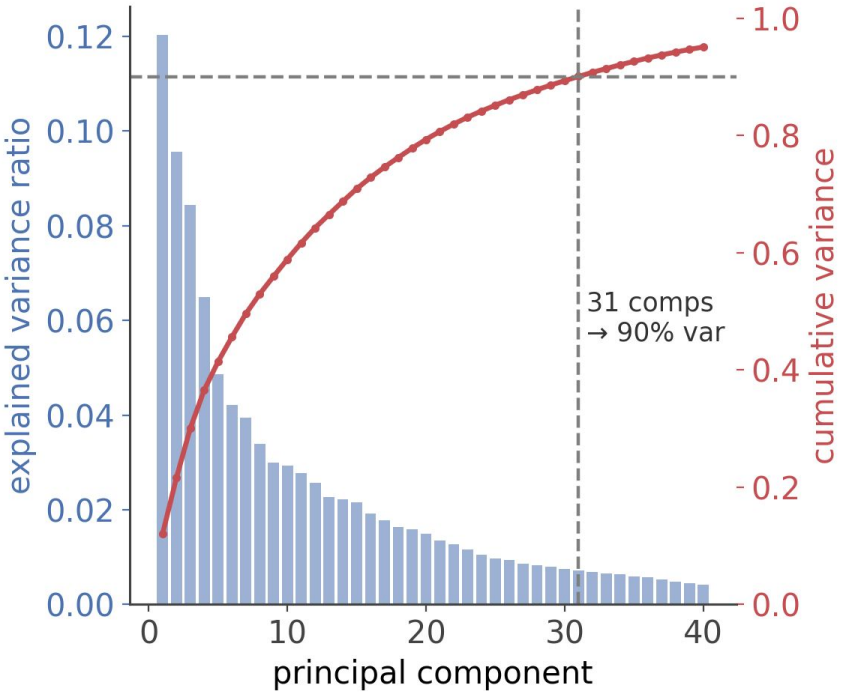
The algorithm:

1. Centre the data
2. Compute the covariance matrix Σ
3. Eigendecompose Σ
4. Keep the top-k principal components.

In practice done by SVD of the data matrix

Largest eigenvalue → greatest variance → the first principal component.

Scree Plot for Choosing Number of Principal Components



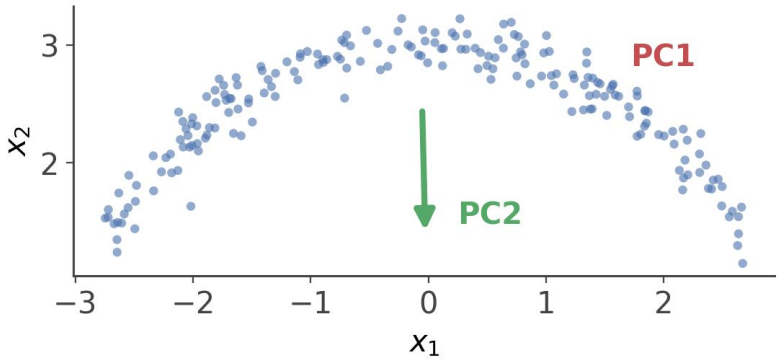
Each principal component has an explained variance ratio, defined:

$$\mathrm{EV}_i = \frac{\lambda_i}{\sum_{k=1}^{\mathrm{rank}(\Sigma)} \lambda_k}$$

The cumulative sum shows how much *information* we keep as we add components.

Fit PCA on the training set only, then apply the same transform to test data.

When PCA is Not Enough



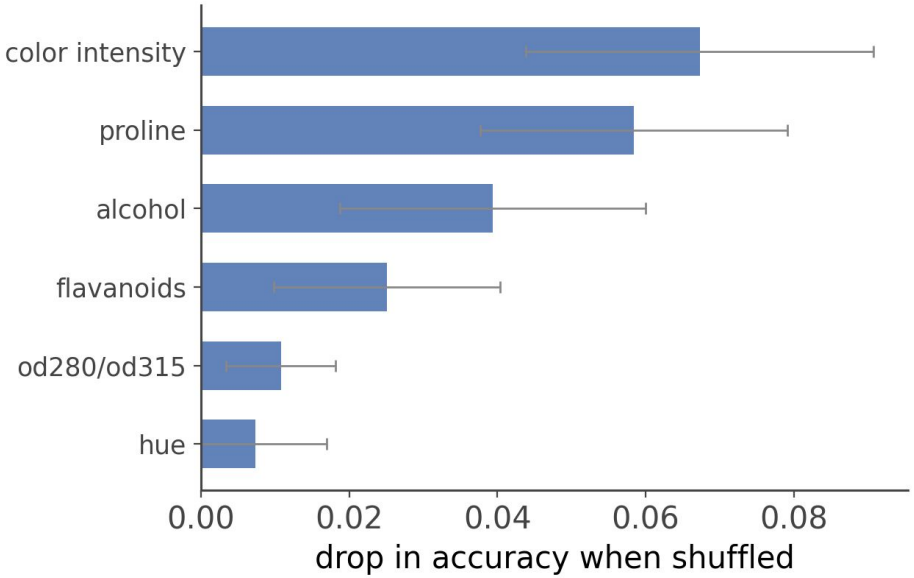
PCA assumes that interesting structure lies along straight, high-variance directions.

This is true for roughly linear or Gaussian shaped data.

On strongly nonlinear data, straight axes can’t follow the curve, so a few components lose the structure.

Nonlinear alternatives: kernel PCA, or autoencoders (neural networks that learn a nonlinear compression).

Feature Importance



To test whether a trained model is over-depending on a particular feature, consider the following experiment:

1. Pick one feature
2. Shuffle its column so it becomes noise, leaving every other feature untouched
3. Measure how far accuracy falls when evaluating model on the test set.

A large drop means the model relied on that feature, while small/no change means it didn't. This ranks features by how much the finished model actually depends on each one.

k-Fold Cross Validation

Train/Validation/Test Splits Can be Lucky

We saw before that we split our dataset to contain data we want our model to train on, and *held out* data that we use as a way to evaluate our model in deployment.

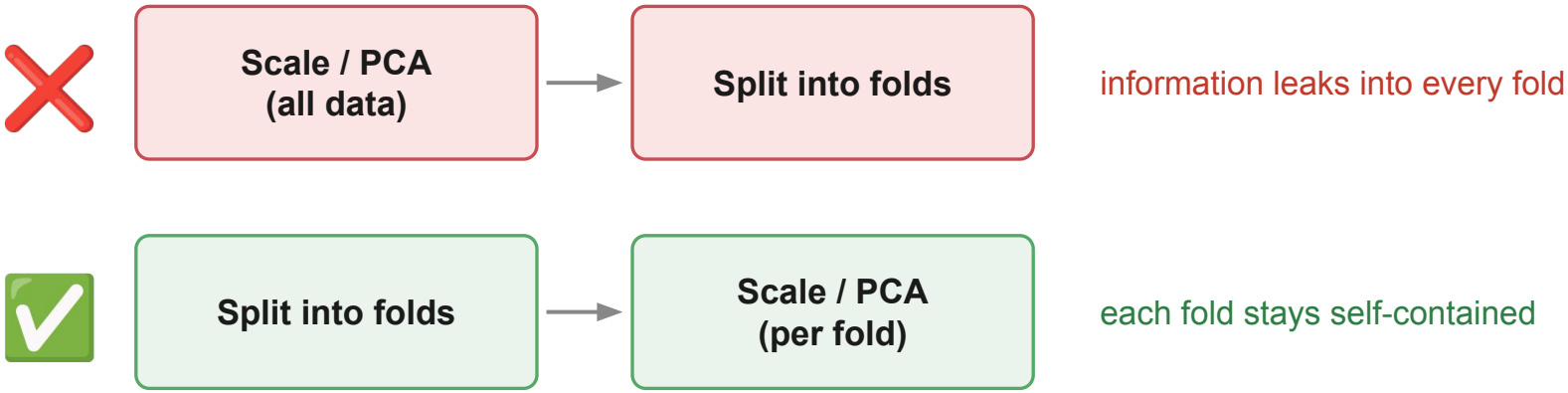
A single split is noisy, where, a lucky or unlucky split can result in differing training performance.

Cross Validation

k-fold cross-validation splits the training data into k parts. Each part takes a turn as validation while the rest train.

The final performance is taken by averaging the k scores. This yields a far more stable metric of performance and generalisation.

A Warning on Data Leakage



Leakage is when information from outside the training fold sneaks in, inflating your score. Scaling or running PCA on the whole dataset before cross-validation means that each fold has peeked at its own validation data.

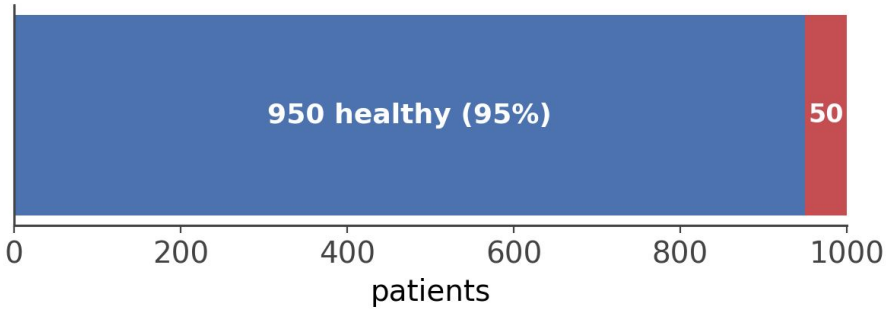
Safe Pipeline

Inside cross-validation it is refit on each fold, so scalers/PCA never see validation data.

Confusion? 🤔

Accuracy as a Metric of Classification

Imagine screening for a disease that affects 5% of patients. Our dataset would like:



Simplest way to score the performance of a classifier is via the accuracy score:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{total number of predictions}}$$

Accuracy Can Lie…

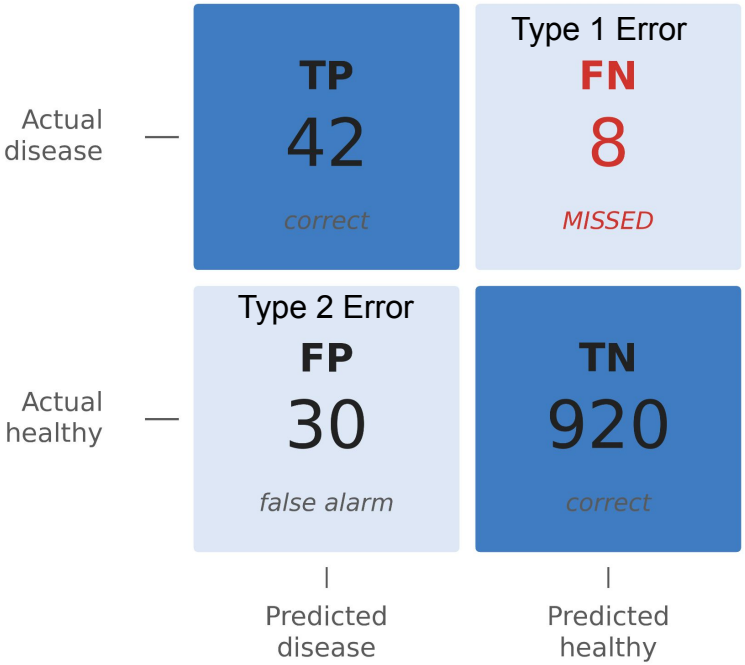
Consider a model that predicts "healthy" for all patients.

Such a model is right 95% of the time, simply because only 5% are ill.

Its accuracy looks excellent at 95%! But catches nobody that has the disease.

Accuracy just counts correct predictions, so on imbalanced data it rewards getting the easy majority right and hides total failure on the rare class that matters.

Confusion matrix & precision / recall



Break predictions into **T**rue **P**ositive / **F**alse **P**ositive / **F**alse **N**egative / **T**rue **N**egative.

Precision = TP / (TP+FP)
of those flagged, how many truly had it.

Recall = TP / (TP+FN)
of those who had it, how many we caught.

In screening, **false negatives** are the dangerous error, so, we tend to prioritise recall when designing tests.

The F1 score captures both precision and recall:

$$F_1 = \frac{2\mathrm{TP}}{2\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

Time for some code…